Procedural Terrain – Reflective Report

Overview

I built a real-time valley landscape with a small lake in DirectX, using C++ and HLSL. The terrain is a tessellated plane displaced by a heightmap, lit by a sun that moves through a day/night cycle, with shadow mapping and a post-processing pass on top.

The aim was to understand the Graphics pipeline rather than using an engine, so I wrote each stage myself: vertex, hull, domain and pixel shaders, render-to-texture for the shadow, post processing passes and the C++ side that creates and binds every buffer and texture. The heightmap displaces the terrain in both the shadow and scene passes, so shadows are cast by the hills rather than a flat plane.

What I'm most proud of

Heightmap displacement with tessellation. Rather than loading a dense mesh, I sent a low-polygon plane to the GPU and let the hull shader subdivide it. The domain shader then samples the heightmap at each new vertex and pushes it upwards. This keeps the vertex buffer small and puts the detail where the GPU can generate it. Normals have to be rebuilt after displacement, so I calculate them from neighbouring heightmap samples, which keeps the lighting consistent with the new shape.

Shadow mapping. Each frame starts with a depth-only pass rendered from the sun's point of view into a depth texture. In the main pass, every pixel is transformed into the light's space and its depth compared against that texture to decide whether it is lit. The terrain has to be tessellated and displaced in both passes, otherwise the shadows would be cast by a flat plane rather than the hills.

Day/night cycle. The sun's direction rotates over time, and the light colour and ambient level are blended between day and night values. Because the shadow pass is built from the light direction, the shadows sweep across the valley as the sun moves. That made the cycle a good stress test for the whole pipeline: any error in the light matrices showed up immediately.



Challenges and how I debugged them

Most of my debugging was done in RenderDoc. Graphics bugs rarely crash; they just draw the wrong thing. Capturing a frame let me step through each draw each call and inspect the textures, constant buffers and shader inputs at every stage.

Shader resource slot conflicts. Once the shadow map was added, the terrain started showing the wrong texture. RenderDoc showed two resourced being bound to the same register, so one silently replaced the other. The fix was to give every texture a fixed, documented slot and match it between the C++ binding code and the HLSL register declarations.

A broken light matrix, at certain points in the day/night cycle the shadows collapsed or vanished. I traced it to the light's view matrix: as the sun's direction approached the "up" vector used to build it, the matrix degenerated. Handling that case explicitly made the cycle stable through the whole rotation.

Gamma and the heightmap. The terrain came out flatter and more stepped than the source image suggested. The heightmap was being treated as an sRGB colour texture, so its values were gamma-corrected before the domain shader read them. Heights are data, not colours, so loading it in a linear format fixed the shape.

Changing scope. I originally planned terrain that scrolled endlessly. It caused seams and sampling problems, and it was eating at the time I needed for lighting and shadows. I pivoted to switching between heightmaps with a button. It was the right call for the deadline, but it is the feature I would most like to revisit.

What I would do better

Scale tessellation with distance. My tessellation level was the same everywhere, so distant hills got as many triangles as the ground at the camera's position. Setting the factor hull shader from camera distance would cut the triangle count with no visible loss.

Make resource binding impossible to get wrong. The slot conflict was a process problem, not a one-off bug. I would define every slot once in a header shared by C++ and HLSL and wrap binding in a small class that asserts in debug builds when a slot is reused.

Better shadows. A single shadow map covering the whole valley trades resolution for coverage, which shows as blocky edges close up. Percentage-close filtering would soften the edges cheaply. Cascaded shadow maps would give sharp shadows near the camera.

Measure Performance, I judged performance by eye, next time I would add GPU timestamp queries around each pass, so decisions like the tessellation level are backed by numbers.

Return to infinite terrain. Generating the heightmap with noise in a compute shader, in tiles around the camera, would remove the seams that defeated my first attempt and help bring the project back to the original goal.

Conclusion

This project taught me how a frame is actually built, from the c++ that binds a texture to the shader that reads it. The hardest problems were not the techniques themselves but the places where stages meet. RenderDoc turned those from guesswork into evidence. The main lesson I carry forward is to design systems, so these mistakes are hard to make in the first place.